

JAVA 25

Introduzione a Java 25

Java 25 è stato rilasciato il **16 settembre 2025** ed è una release **LTS** (Long Term Support), cioè con supporto a lungo termine, come lo furono le versioni 8, 11, 17 e 21. Le versioni LTS ricevono aggiornamenti e supporto per anni, quindi sono quelle che le aziende adottano in produzione.

Un concetto chiave dell'introduzione è la differenza tra **funzionalità "consegnata" (delivered/final)** e **funzionalità "in anteprima" (preview)**:

- Una funzionalità **consegnata** è stabile e definitiva: non verrà rimossa.
- Una funzionalità **preview** è sperimentale e può essere rimossa o modificata in una release successiva.

Tutti gli argomenti trattati nel corso sono funzionalità *consegnate* in Java 25. Molte di esse però erano nate come preview in versioni precedenti e sono state finalizzate col tempo. Ecco la panoramica:

Funzionalità	Introdotta/Finalizzata
Unnamed Variables and Patterns (variabili e pattern senza nome)	preview in Java 21, finalizzata in Java 22
Markdown Documentation Comments	Java 23
Module Import Declarations	Java 25
Compact Source Files & Instance Main Methods	preview in Java 21, finalizzata in Java 25
Flexible Constructor Bodies	Java 25
Scoped Values	Java 25
Stream Gatherers	finalizzata in Java 24
Ogni funzionalità corrisponde a una JEP (JDK Enhancement Proposal), cioè la proposta ufficiale di miglioramento del JDK (es. Markdown è la JEP 467).	

Unnamed Variables and Patterns (variabili e pattern senza nome)

Definizione. Sono un meccanismo, indicato dal carattere **underscore** `_` (trattino basso), per dichiarare esplicitamente che una variabile *non verrà usata*. L'obiettivo è rendere chiaro l'intento di non-utilizzo: il codice diventa più pulito, leggibile e manutenibile, e gli strumenti di analisi statica non segnalano più warning per "variabile inutilizzata".

Un punto importante: le variabili senza nome si applicano solo alle **variabili locali**, non alle variabili di istanza. In particolare valgono per: dichiarazioni di variabili locali, intestazioni di cicli `for`, parametri delle risorse in `try-with-resources`, parametri nelle clausole `catch`, e parametri delle **lambda**.

I **pattern senza nome** invece permettono di ignorare campi specifici quando si usa il **pattern matching**, e sono particolarmente utili nella **decostruzione dei record** e negli `switch`.

Esempi visti nel codice

1. Ciclo for potenziato (usato solo per effetti collaterali). Se itero su un array di nomi (April, Brendan) ma dentro il ciclo non uso mai il nome, invece di dare un nome all'identificatore uso `_`. Chi legge capisce subito: "questo ciclo serve solo per il suo effetto collaterale, non mi interessano i valori".

```
for (var _ : names) {  
    // faccio qualcosa che non usa il valore  
}
```

2. Lambda. Stessa idea: se la lambda riceve un parametro (name, oppure un indice `i`) che non usa, lo si sostituisce con `_`.

3. try-with-resources ed eccezioni. Se dichiaro un `BufferedReader` ma non uso mai quel riferimento nel blocco `try`, uso `_`. Idem nel `catch`: se non uso il riferimento all'eccezione, `catch (IOException _)`.

4. Switch con pattern. Se scrivo `case String s ->` ma non uso `s` sul lato destro della freccia, scrivo `case String _ ->`. Così esprimo che *mi interessa solo il tipo* (è una stringa, è un intero), non il valore. Nota: il `default` resta necessario per garantire l'**esaustività** dello switch (altrimenti errore di compilazione).

5. Decostruzione di record. Dato un record `Person(String name, int age)`:

```
if (somebody instanceof Person(String name, int _)) {  
    // uso solo name, l'età non mi interessa  
}
```

Con i **record annidati** funziona in profondità. Con `Person(String name, int age, Address(String city, String country))`, se mi interessano solo `name` e `country`:

```
if (obj instanceof Person(String name, int _, Address(String _, String country))) { ... }
```

L'underscore rende esplicito che età e città non vengono usate.

Markdown Documentation Comments

Definizione. Da Java 23 è possibile scrivere la documentazione delle API Java (i JavaDoc) usando la sintassi **Markdown** invece dell'HTML.

Il perché. L'HTML è potente e standardizzato (usato dai browser dagli anni '90) ma è **noioso da scrivere e difficile da leggere** a mano. Il Markdown, inventato nel 2004 da John Gruber (con Aaron Swartz), è un linguaggio di markup leggero pensato per la **massima leggibilità** e facilmente convertibile in HTML. Un dettaglio: la descrizione originale di Gruber non è del tutto univoca, quindi processori diversi possono rendere lo stesso Markdown in modi leggermente diversi. Comodamente, il Markdown permette comunque di inserire tag HTML per funzioni non coperte nativamente.

Sintassi Markdown vista nell'esempio

- **Intestazioni:** # = <h1>, ## = <h2>, ### = <h3>, ecc.
- **Commenti:** si usa il commento HTML <!-- ... -->.
- **Elenchi non ordinati:** riga che inizia con - (trattino + spazio). I **sottoelenchi** si ottengono con 4 spazi di indentazione prima del trattino.
- **Elenchi ordinati:** numero + . + spazio (es. 1.). Anche qui 4 spazi per i sottoelenchi (il numero effettivo conta poco: l'indentazione determina l'annidamento).
- **Elenchi di task:** come i non ordinati ma con [X] (completato) o [] (incompleto).
- **Linea orizzontale:** tre trattini - - -.
- **Enfasi:** corsivo = *testo* o _testo_; grassetto = **testo** o __testo__; grassetto+corsivo = ***testo***; barrato = ~testo~ (tilde).
- **A capo nello stesso paragrafo:** due spazi a fine riga + invio. Per un **nuovo paragrafo** serve una riga vuota.
- **Link:** [testo](url) — il testo tra parentesi quadre, la destinazione tra tonde. Con #ancora si salta a una sezione interna del documento.
- **Immagini:** come i link ma con ! davanti: ![testo alternativo](percorso).
- **Codice inline:** racchiuso tra backtick `codice`.
- **Blocco di codice:** tre backtick sulla riga prima e dopo la sezione.
- **Citazioni (blockquote):** ogni riga inizia con > (anche le righe vuote); >> per citazioni annidate.
- **Tabelle:** si usano le barre verticali | per le colonne e i trattini per separare l'intestazione dai dati.

Sia IntelliJ che VS Code offrono un'**anteprima affiancata** (file Markdown a sinistra, resa HTML a destra), e GitHub rende automaticamente i .md come HTML.

Module Import Declarations (dichiarazioni di importazione dei moduli)

Definizione. Nuova in Java 25. Con la sintassi `import module M;` si importano **in un colpo solo** tutti i tipi pubblici di primo livello (classi e interfacce) dei pacchetti **esportati** dal modulo M, invece di scrivere molte `import` individuali.

Vantaggi e caratteristiche:

- Riduce il "rumore" delle importazioni in cima al file.
- Il codice **non deve far parte di un modulo** (non serve un file `module-info.java`).
- È una funzionalità di **risoluzione dei nomi in fase di compilazione**: allarga solo il percorso di ricerca dei tipi, **non gonfia** il file di classe (non copia intere librerie). Quindi nessun impatto sulle dimensioni del bytecode.

Avvertenza — conflitti di nomi. Importando più moduli si possono avere collisioni. Esempio tipico: `List` — quella di `java.awt` o quella di `java.util`? Se il compilatore trova due `List` dà errore *"reference to List is ambiguous"*. Come si risolve:

- Si importa **esplicitamente il tipo specifico** (`import java.util.List;`). L'import di un singolo tipo ha **precedenza** sull'import di un modulo.

- Oppure si usa il **nome pienamente qualificato** (`java.util.List`).

Esempio nel codice

Un programma legge da un file `data.txt` una lista di frutti, poi crea una **mappa "lunghezza parola → numero di parole con quella lunghezza"** e infine trova la parola più lunga.

- Legge tutte le righe in una `List<String>`.
- Fa `stream() + collect(groupingBy(...))` raggruppando per lunghezza della riga → mappa tipo `{4=2, 5=1, 6=2, 10=1}` (pera e kiwi = 4 lettere, mela = 5, banana ×2 = 6, fragola = 10).
- Con un secondo stream e `max(comparingInt(...))` trova la parola più lunga (fragola), usando `.orElse(...)` perché il risultato è un `Optional`.

La dimostrazione: il file inizialmente ha 6 righe di `import` (righe 4–9). Sostituendole tutte con la singola riga `import module java.base;`, tutti gli errori spariscono — **6 import → 1**. Poi, aggiungendo `import module java.desktop;`, compare l'ambiguità su `List`, risolta con `import java.util.List;` (che ha la precedenza). Viene anche anticipato che nei **compact source files** il modulo `java.base` è importato *implicitamente*.

Compact Source Files e Instance Main Methods

Definizione. Consegnata in Java 25 (era preview in Java 21 col nome "unnamed classes"). Permette di scrivere piccoli programmi **senza dichiarare esplicitamente una classe** e usando un metodo **main di istanza** invece del classico `public static void main(String[] args)`.

Caratteristiche chiave:

- Ogni compact source file importa **implicitamente** `import module java.base` all'inizio → meno `import` da scrivere.
- Nuova classe di utilità `java.io.IO` per input/output semplice da console (basata su `System.in/System.out`), es. `IO.println(...)`.
- Pensata per **principianti** e per **prototipazione rapida**.

Come funziona (dall'esempio)

Si parte da un "Hello World" tradizionale (con `package`, `class`, `public static void main`, `System.out.println`) e lo si converte in compact:

- **Niente package:** un compact source file finisce nel *pacchetto predefinito (unnamed)*, quindi non può avere un package esplicito.
- **Niente class:** la classe è "senza nome" dal punto di vista del sorgente, ma sul disco **prende il nome dal file** (es. file `CompactHelloWorld.java` → classe `CompactHelloWorld`).
- **main di istanza:** si tolgono `public static` e il parametro `String[] args` → `void main() { IO.println("..."); }`.

Cosa succede "sotto il cofano":

- Se dichiaro una **variabile** fuori dal `main` (es. `int x = 5;`), diventa una **variabile di istanza** della classe generata.
- Se aggiungo un **metodo** `m()`, diventa un metodo di istanza.
- Se aggiungo una **classe interna**, sul disco appare come *inner class* (es. `CompactHelloWorld$Inner.class`).
- Il disassemblatore mostra: la classe `CompactHelloWorld`, la variabile di istanza `x`, un **costruttore no-arg** generato automaticamente, e i metodi di istanza `main` e `m`. Tutto ciò che è nel file viene incapsulato in quella classe che prende il nome dal file.

Overload del main con argomenti. Si può anche dichiarare `void main(String[] args)`. Il *launcher* userà questa versione se esiste. Nell'esempio: se `args.length == 0` si chiama il `main` senza argomenti, altrimenti si stampa quanti argomenti sono stati passati e li si itera. Eseguendo con argomenti (`ABC, Def`), la lunghezza non è zero, quindi viene eseguito il ramo che stampa gli argomenti.

Flexible Constructor Bodies (corpi flessibili dei costruttori)

Definizione. Nuova in Java 25. **Prima**, la prima istruzione di un costruttore doveva obbligatoriamente essere una chiamata a `super(...)` o `this(...)`. **Ora** si può inserire del codice **prima** di quella chiamata.

Struttura del costruttore: si divide in tre aree —

1. **Prologo:** codice *prima* di `super/this` (con delle limitazioni).
2. La chiamata a `super/this`.
3. **Epilogo:** codice *dopo*.

Cosa si può fare nel prologo:

- **Validare gli input** prima di passare al costruttore genitore.
- **Inizializzare le variabili di istanza** prima della `super`.

Cosa NON si può fare nel prologo (perché l'oggetto è solo parzialmente costruito):

- **Chiamare metodi di istanza.**
- **Leggere** le variabili di istanza (si possono solo inizializzare/scrivere).
- Il riferimento `this` è utilizzabile *solo* per inizializzare i campi consentiti, non per invocare metodi o leggere campi.

Se non c'è una `super/this` esplicita, Java inserisce automaticamente `super();` (a patto che la superclasse abbia un costruttore no-arg accessibile).

Due grandi vantaggi:

1. **Validazione al posto giusto e nel costruttore stesso** (non delegata a un metodo helper esterno).
2. Risolve il classico problema del **costruttore della superclasse che chiama un metodo sovrascrivibile** — che nella sottoclasse leggerebbe campi ancora non inizializzati (rischio di `null`).

Esempio nel codice

Classe base `Animal` con campo `name`, il cui costruttore valida (`Objects.requireNonNull`, controllo stringa vuota → eccezione), inizializza `name`, poi chiama un metodo **polimorfico** `logDetails()`.

Sottoclasse `Dog` con campo `breed`.

Comportamento pre-Java 25 (super come prima riga): creando un `Dog("Rex", "terrier")`, quando `Animal` chiama `logDetails()`, per polimorfismo esegue la versione di `Dog` — ma `breed` **non è ancora inizializzato** → stampa `"breed è null"`. È un bug tipico.

Con i flexible constructor bodies: nella sottoclasse si può inizializzare `breed` **prima** di chiamare `super`. Così quando `super` (e quindi `logDetails()` di `Dog`) viene eseguito, `breed` è già valorizzato → stampa correttamente `"terrier"`. In più, spostando `super` si può **validare breed prima** di validare il nome nel genitore: se `breed` non è valido si fallisce subito, senza sprecare lavoro.

Le limitazioni dimostrate: nel prologo non si può chiamare `this.logDetails()` (metodo di istanza) né leggere `this.breed` (lettura di campo) — solo scrivere/inizializzare.

Viene infine mostrato che, convertendo in *instance main*, l'`instance main` richiede un **costruttore no-arg**, e la regola sulla `super` inserita automaticamente resta valida a prescindere dalle modifiche a prologo/epilogo.

Scoped Values (valori con scope)

Motivazione

Il problema: hai informazioni di contesto (es. **user ID**, **request ID**) da condividere tra molti metodi lungo lo stack di chiamate.

- **Passarle come parametro** a ogni metodo appesantisce il codice e costringe a modificare tutte le firme.
- Tradizionalmente si usava **ThreadLocal**, che però ha tre problemi: (1) il binding è **mutabile** (sovrascrivibile per sbaglio); (2) rischio di **leak** (se dimentichi di fare `remove()`, i dati "trapelano" oltre lo scope voluto); (3) **non si adatta bene ai thread virtuali** e ai nuovi modelli di concorrenza.

Definizione di Scoped Value. Java 25 introduce lo **Scoped Value**: pensalo come una mappa in cui il valore con scope è **legato (bound)** al valore di contesto da condividere. Caratteristiche:

- Il binding vale per uno **scope** = un **blocco di esecuzione**.
- Lo scope è **dinamico, non lessicale**: segue lo **stack di chiamate**, non le parentesi graffe.
- Il binding è di **sola lettura** dentro lo scope (non si può sovrascrivere per errore → risolve il problema #1 di `ThreadLocal`).
- Quando il blocco finisce, il valore viene **automaticamente svincolato** (→ risolve i leak, problema #2).

- Si può fare **rebinding in uno scope annidato**: il binding interno sovrascrive *temporaneamente* quello esterno, che ritorna visibile quando lo scope interno termina.
- Idealmente gli oggetti a cui ci si lega dovrebbero essere **immutabili** (es. una `String` anziché uno `StringBuilder`).

ThreadLocal (esempio di codice) — per mostrare i problemi

```
static final ThreadLocal<String> userId = new ThreadLocal<>();
```

Lo scope è definito **per convenzione**: `set( ... )` inizia, `remove( )` finisce. Tutti i metodi chiamati tra `set` e `remove` "vedono" lo user ID (scope dinamico). Un metodo chiamato **dopo** `remove( )` legge `null` (corretto).

I due problemi dimostrati:

1. **Mutabilità**: basta un `set( "xyz" )` da qualche parte per cambiare accidentalmente il valore ovunque a valle.
2. **Leak**: se si commenta/dimentica il `remove( )`, un metodo che *non dovrebbe* vedere lo user ID lo vede lo stesso.

Scoped Values (esempio di codice) — la soluzione

```
static final ScopedValue<String> userId = ScopedValue.newInstance();
```

```
ScopedValue.where(userId, "SK001")
```

```
.run() -> { /* blocco = scope */ };
```

- **newInstance()** crea lo scoped value.
- **where(userId, "SK001")** lega lo user ID a un valore.
- **run(runnable)** definisce lo scope ed esegue il blocco **sul thread corrente** (non avvia un nuovo thread).
- Si legge con **userId.get()** (spesso con `.orElse( ... )`).

Tutti i metodi chiamati dentro `run` (e i metodi da essi chiamati) vedono lo user ID — è **scope dinamico**. Il valore è **read-only**: non esiste un `set`, se provi a impostarlo il compilatore non te lo permette.

Cosa dimostra l'esempio:

- Un metodo chiamato *fuori* dal `run` **non vede** lo user ID (niente leak): il binding è svincolato automaticamente a fine scope.
- **Rebinding**: rifacendo `where(userId, "SK002")` in un nuovo scope si ottiene il nuovo valore; fuori resta `null`.
- **Scope annidato**: dentro lo scope esterno (`SK001 / "outer"`) si apre un secondo `where(userId, "inner").run( ... )`. Dentro l'annidato l'user ID vale "inner" per tutti i metodi chiamati lì; **uscendo** dall'annidato, il binding esterno "outer" torna visibile ed è ripristinato.

Riepilogo API: **crei** con `newInstance( )`, **leggi** con `where`, **definisci lo scope** con `run`, **accedi** con `get/orElse`. Gli scope annidati permettono il rebinding temporaneo.

Stream Gatherers

Introduzione

Definizione. I **Gatherers** (finalizzati in Java 24) permettono di definire **operazioni intermedie personalizzate** sugli stream. Prima si era limitati a un "menu fisso" di operazioni intermedie (`map`, `filter`, `distinct`, ...), il che rendeva scomodi certi pattern.

Esempi di pattern scomodi *prima* dei gatherers:

- **Distinct by key:** il `distinct()` standard usa l'intero oggetto per l'uguaglianza; spesso vuoi l'unicità basata su *un solo campo* (es. l'email).
- **Batching/chunking:** raggruppare elementi in gruppi di `N` — prima bisognava raccogliere in una lista e post-processare con dei loop fuori dalla pipeline.

L'analogia chiave: così come `collect(Collector)` è il punto di estensione per le **operazioni terminali**, `gather(Gatherer)` è il punto di estensione per le **operazioni intermedie**. In altre parole: *i Gatherer stanno alle operazioni intermedie come i Collector stanno alle terminali*.

Cosa consentono i gatherers:

- Esaminare ogni elemento mentre scorre.
- Mantenere uno **stato personalizzato** tra gli elementi (unicità per campo, rimozione di duplicati consecutivi, totali progressivi...).
- Fare **cortocircuito** (fermarsi prima → utile per trasformare **stream infiniti** in risultati finiti).
- **Emettere 0, 1 o molti** elementi per ogni input (il `filter` standard può emettere solo 0 o 1).

Anatomia — la forma generica `Gatherer<T, A, R>`:

- **T** = tipo dell'elemento in **ingresso** (upstream).
- **A** = tipo dello **stato mutabile** tracciato mentre gli elementi scorrono.
- **R** = tipo dell'elemento in **uscita** (downstream).

Le due modalità di creazione:

- **`Gatherer.ofSequential(...)`** — valutazione **solo sequenziale** (quando lo stream non può essere partizionato senza cambiare il risultato). Non usabile in parallelo. Parametri: *initializer*, *integrator*, *finisher* (opzionali quelli che non servono).
- **`Gatherer.of(...)`** — valutazione **parallelizzabile** (quando lo stream può essere partizionato). Ha in più un **combiner**. Se fornisci il combiner, il gatherer è **stateful** (unirà due stati).

Le quattro forme funzionali che compongono un gatherer:

1. **Initializer** (un `Supplier`): crea la **memoria di stato** (es. `HashSet::new`). Opzionale — se il gatherer è *stateless*, lo stato è `Void/null`.
2. **Integrator:** la **logica personalizzata**, eseguita per **ogni elemento**. Aggiorna lo stato, può spingere elementi a valle, e **restituisce un boolean** (`true` = continua, `false` = ferma/cortocircuita). Riceve (*state*, *element*, *downstream*); `downstream.push(...)` restituisce un boolean (`false` = il downstream rifiuta altri elementi).

3. **Combiner** (un `BinaryOperator`): **unisce due stati** quando lo stream è valutato in parallelo (sinistra + destra → uno solo). Presente solo nei gatherer stateful/paralleli.
4. **Finisher** (un `Consumer`): **svuota/emette l'output residuo** alla fine. Riceve (*state*, *downstream*); non restituisce nulla.

Built-In Gatherers (raccolgitori integrati)

- **`Gatherers.windowFixed(size)`** — **finestre fisse** (chunk), senza sovrapposizione; l'ultima finestra può essere più corta. Con `size=3` su `1..8` → `[1, 2, 3]` `[4, 5, 6]` `[7, 8]`.
- **`Gatherers.windowSliding(size)`** — **finestre scorrevoli**, con sovrapposizione. Con `size=3` su `1..8` → `[1, 2, 3]` `[2, 3, 4]` `[3, 4, 5]` `[4, 5, 6]` `[5, 6, 7]` `[6, 7, 8]`.
- **`Gatherers.scan(supplier, function)`** — **totale progressivo (running total)**: emette un risultato per *ogni* elemento (diverso da `reduce`, che dà un solo risultato finale). Con valore iniziale 0 e somma, su `1, 2, 3, 4, 5` → `1, 3, 6, 10, 15`. ($0+1=1$, $1+2=3$, $3+3=6$, $6+4=10$, $10+5=15$.)

Distinct By Key (esempio)

Obiettivo: dato un elenco di `User(id, email, purchases)` (un **record**, che genera automaticamente costruttore canonico, accessor, `equals/hashCode/toString`), tenere il **primo utente per ogni email distinta**.

- **Perché `distinct()` non basta**: usa *tutti* i componenti del record per l'uguaglianza (`id`, `email`, `purchases`). Due utenti con la stessa email ma `id` diverso risultano "diversi" → l'email compare due volte. Sbagliato.
- **Vecchio metodo (senza gatherer)**: un `HashSet` esterno alla pipeline + `filter(u -> seen.add(u.email()))`. `add` restituisce `true` se l'email non era presente (→ elemento tenuto), `false` se già vista (→ scartato).
- **Con gatherer**: un metodo custom `distinctByKey(keyExtractor)` che ritorna il gatherer. Il bello: per cambiare chiave (`email` → `purchases`) basta passare un **method reference diverso**.

Struttura del gatherer `distinctByKey`, generico `<T, K>` (`T` = `User`, `K` = tipo della chiave: `String` per email, `Integer` per `purchases`), stato = `Set<K>` (le chiavi viste):

- **`Gatherer.ofSequential(...)`** (niente combiner → niente parallelo).
- **Initializer**: `HashSet::new`, eseguito una volta per pipeline.
- **Integrator (ofGreedy)**: elabora ogni elemento subito. Estrae la chiave; `seenKeys.add(key)` → se `true` (prima volta) fa `downstream.push(element)` (emette); se il push ritorna `false` significa che il downstream vuole fermarsi → si restituisce `false` (cortocircuito). Se la chiave era già presente, non emette. `true` = continua, `false` = stop.
- **Finisher**: non necessario qui (viene omissso).

Cosa fa un integratore *greedy*: consuma tutto l'upstream finché non è esaurito o finché il downstream non rifiuta.

Drop Consecutive Duplicates By Key (esempio)

Obiettivo: eliminare solo i **duplicati consecutivi** per chiave. Con email [a, a, b, c, c, a] (id 1..6) si scartano l'id 2 (a consecutiva ad a) e l'id 5 (c consecutiva a c); l'id 6 (a) **si tiene** perché la precedente (c) è diversa. Questo `distinct()` non lo può fare.

Struttura: come prima, generico `<T, K>`, ma con una **classe locale State** che mantiene la memoria tra elementi:

- Un flag `isFirstElement = true` (per far passare sempre il primissimo elemento).
- Un campo `lastKey`.
- **Initializer:** crea la `State`, una volta per pipeline (stessa istanza riusata per tutta la raccolta).
- **Integrator (greedy):** estrae la chiave; l'`if` emette l'elemento se è *il primo oppure se la chiave è diversa dalla precedente*. Quando emette, aggiorna `lastKey`, mette `isFirstElement = false`, fa `push` a valle e restituisce il suo boolean. Se la chiave è uguale alla precedente, non emette e restituisce `downstream.isRejecting()` (o comunque prosegue/ferma di conseguenza).

Cambiando il method reference (email → purchases) si riusa lo stesso `gatherer`.

Short Circuit (esempio)

Obiettivo: sommare interi ed emetterli **finché il totale supera 10**, poi **fermarsi**. Con [2, 2, 2, 3, 5, ...]: 2+2+2+3 = 9 (ok), +5 = 14 → stop. Output: 2, 2, 2, 3.

Qui si usa **Gatherer.of(...)** con l'**integratore Integrator.of(...)** (non `ofGreedy`):

- Differenza chiave: l'integratore **greedy** non va *mai* in cortocircuito da solo (consuma tutto finché upstream si esaurisce o il downstream rifiuta). L'integratore **non-greedy (of)** può **cortocircuitare da solo** — cioè smettere di consumare l'upstream in anticipo per **sua** decisione logica.
- Stato: una classe locale con un campo `sum` (serve una classe perché una variabile locale usata nella lambda dovrebbe essere `final`/effectively final, quindi non si può mutare direttamente).
- Logica: `nextTotal = state.sum + element`. Se `nextTotal > 10` → **return false** (cortocircuito). Altrimenti aggiorna `state.sum`, fa `push` a valle e continua.

Distinzione importante evidenziata: `downstream.push` che ritorna `false` è il *downstream* che rifiuta; il **cortocircuito vero** è quando *la nostra* logica decide di fermarsi (`return false` per la condizione `> 10`).

Stateless (esempio)

Mostra un `gatherer` **senza stato** e la capacità di emettere **0, 1 o molti** elementi.

- **Metodo filter(predicate) custom:** replica lo stream `filter` (emette 0 o 1 elemento per input). Su [1, 2, 3, 4] con predicato "pari" → 2, 4.
- **Metodo duplicateEach():** duplica ogni input → 1, 1, 2, 2, 3, 3, 4, 4. Questo il `filter` standard **non** lo può fare (emette al massimo 1).

Caratteristiche stateless: firma `Gatherer<T, ?, R>` (il `?` centrale è il tipo di stato **nascosto/inutilizzato**; il primo parametro dell'integratore è `unused` ed è `null`). *Stateless* significa che **ogni decisione dipende solo dall'elemento corrente**, senza ricordare nulla tra elementi → **nessun initializer** necessario. L'analogia usata: un **segnalibro del browser** ti porta sempre alla stessa pagina, indipendentemente da quali pagine hai visitato prima.

Nel `duplicateEach`, l'unica differenza è nell'integratore: `push` dell'elemento due volte, con la logica che se il **primo** `push` ritorna `false` va in cortocircuito e non tenta il secondo. Niente `initializer`, niente `finisher`, niente `combiner` → solo l'integratore.

Finisher (esempio)

Dimostra un algoritmo **dove il finisher è necessario**, e perché **non far trapelare stato mutabile a valle**.

Obiettivo: replicare `windowFixed(3)` su `1..8` → chunk `[1, 2, 3]` `[4, 5, 6]` `[7, 8]`.

L'ultimo chunk `(7, 8)` è più corto della finestra e viene emesso **solo grazie al finisher**.

Struttura: `Gatherer<T, ..., List<T>>`, stato = un `ArrayList<T>` chiamato `buff`.

- **Initializer:** crea `buff`.
- **Integrator (greedy):** aggiunge l'elemento al buffer; quando `buff.size() == windowSize(3)`, prende uno **snapshot** del buffer (`List.copyOf(buff)`) → lista **non modificabile**, lo mette in chunk, **svuota il buffer** e fa `push(chunk)` a valle.
Fondamentale: si emette una **copia**, non il buffer stesso, perché il buffer viene poi svuotato e mutato.
- **Finisher:** quando l'upstream è esaurito, se il buffer **non è vuoto** ne fa una copia (`tail`) e la spinge a valle → è così che `[7, 8]` esce.

Dimostrazione del **leak di stato mutabile**: poiché si emette `List.copyOf(...)` (immutabile), provando a modificare la lista emessa a valle si ottiene `UnsupportedOperationException`. Questo è il comportamento *voluto*: non far trapelare stato mutabile.

Il secondo metodo mostrato è **identico ma con finisher vuoto** (default): niente flush finale → **il chunk [7, 8] viene perso**. Questa è la prova di quando il finisher serve.

Combiner (esempio)

Dimostra un gatherer **parallelo end-to-end**: ogni sotto-task costruisce il proprio stato e il **combiner unisce** gli stati in un unico risultato finale.

Obiettivo: contare gli elementi di uno stream. Range `1..400` (potrebbero essere milioni). Si usa `.parallel()` per creare uno stream parallelo, poi si fa il boxing degli `int` in `Integer` (i gatherer lavorano su stream di oggetti). Il gatherer emette **un solo valore** (400), quindi `findFirst() + orElseThrow()`.

Struttura `Gatherer<T, A, R>` (`T = Integer` in input; `A = una classe locale Counter`; `R = Long`, tipo tradizionale per i conteggi):

- Si usa **`Gatherer.of(...)`** (versione parallela) — quando parallelizzi *devi* fornire il `combiner`.

- **Initializer:** `new Counter()` — eseguito **una volta per sotto-task**. Se lo stream è diviso in 4 sotto-task da 100, si creano **4 contatori**.
- **Integrator:** eseguito **una volta per elemento dentro un sotto-task** (100 volte per un sotto-task da 100). Non usa il valore dell'elemento (`_`), incrementa solo `counter.n`. Non spinge nulla a valle qui.
- **Combiner:** man mano che i sotto-task finiscono, prende due contatori (`left`, `right`), somma `right` in `left` e restituisce `left`. Es: $100+100=200$, $100+100=200$, poi $200+200=400$.
- **Finisher:** prende lo stato finale (400) e lo spinge a valle.

Regola finale ribadita: un **gatherer stateful** costruito con `Gatherer.of(...)` ha bisogno di un **combiner** quando vuoi lavorare con **stream paralleli**.